Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

CS 401/MCS 401 Lecture 6
Computer Algorithms I
Jan Verschelde, 29 June 2026

Scheduling, Caching, Shortest Paths

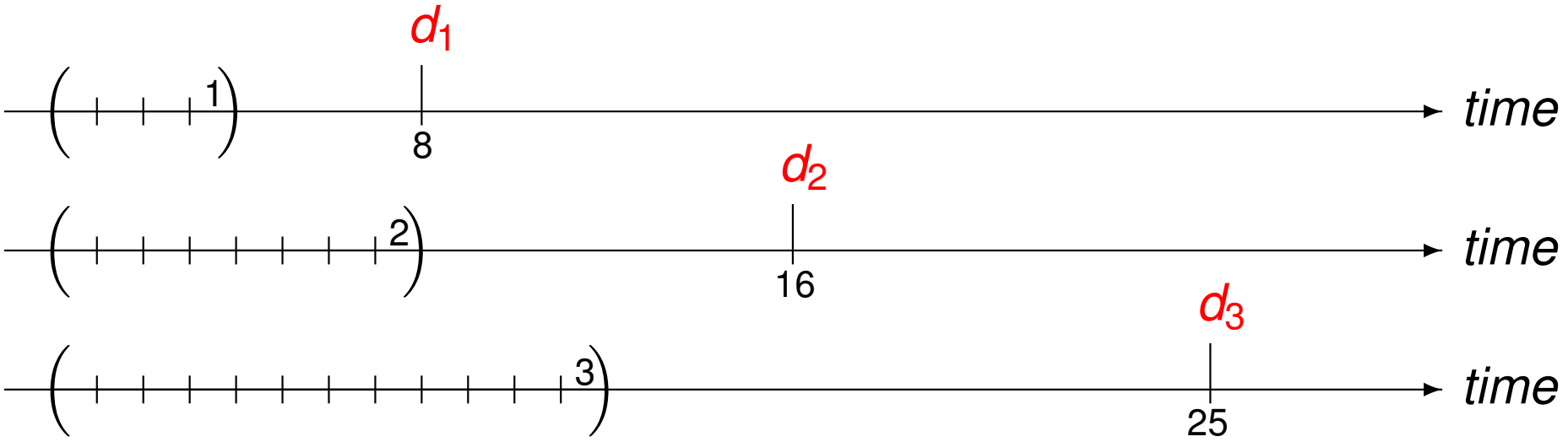
1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

scheduling requests with deadlines

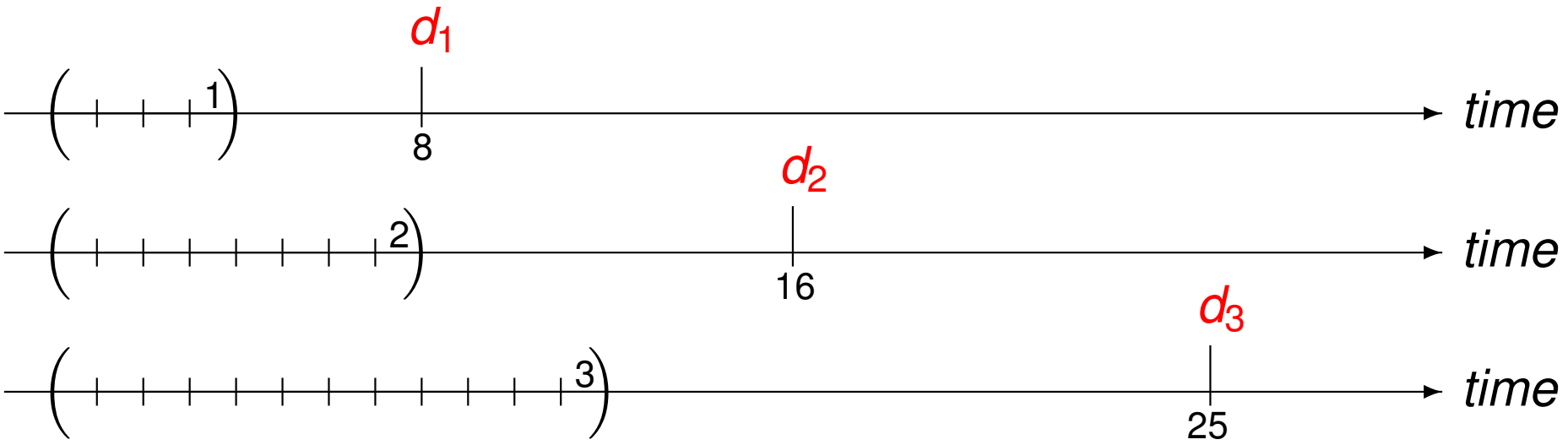
We have one single resource and n requests.

Input: $\{ 1, 2, \ldots, n \}$, a set of n requests,
request i has a deadline d_i
and needs time t_i to complete, for $i = 1, 2, \ldots, n$.

Example of an input of three requests $\{ 1, 2, 3 \}$:
$(t_1, d_1) = (4, 8)$, $(t_2, d_2) = (8, 16)$, $(t_3, d_3) = (12, 25)$.

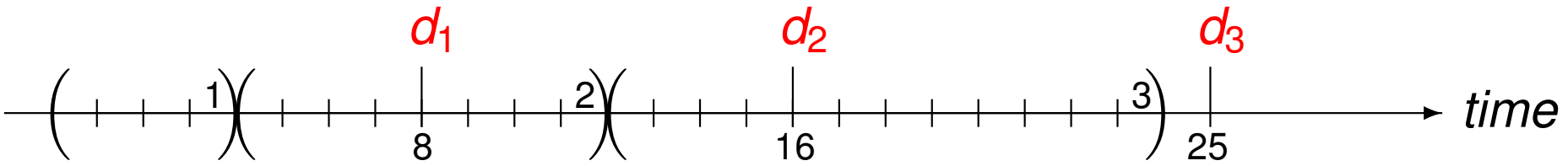


scheduling with deadlines and no overlaps



We want to schedule all requests so that

- there are no overlaps in the time intervals;
- requests should meet their deadlines (or finish not too late).



a meal planning problem

Exercise 1:

Consider a meal planning problem, for one, possibly human, animal.

We have bought a list of food items:

1. every food has an expiration date,
2. the nutritional value of each food determines how long it takes for the animal to go hungry.

The Problem: *Given this list, will food go wasted?*

Does the scheduling with deadlines apply to this problem?

- If no, provide an argument why this problem is different.
- If yes, give an example of an input to the scheduling with deadlines problem which solves this meal planning problem.

Earliest Deadline First algorithm

Input: $\{ 1, 2, \ldots, n \}$, a set of n requests,
request i has a deadline d_i
and needs time t_i to complete, for $i = 1, 2, \ldots, n$.
The requests are sorted by deadlines: $d_1 \leq d_2 \leq \cdots \leq d_n$.

Output: $[s(i), f(i)]$, start and finish time of each request.

$f := 0$
for $i = 1, 2, \ldots, n$ do
    $s(i) := f$
    $f(i) := f + t_i$
    $f := f + t_i$

Is the output schedule optimal?

an optimization problem

Given is a set of n tuples $\{ (t_1, d_1), (t_2, d_2), \ldots, (t_n, d_n) \}$.

The scheduling assigns to each request i a *start time* $s(i)$. Then:

1. The *finish time* is $f(i) := s(i) + t_i$.
2. The *lateness* of request i is $\ell(i) := \max(f(i) - d_i, 0)$.

Definition (1)

For a set of requests $\{ (t_1, d_1), (t_2, d_2), \ldots, (t_n, d_n) \}$ and corresponding start times $A = \{ s(1), s(2), \ldots, s(n) \}$, *the lateness $L(A)$* is $\max_{i=1}^{n} \ell(i)$.

Observe that lateness depends on the schedule A:

$$L(A) = \max_{i=1}^{n} \ell(i) = \max_{i=1}^{n}(\max(f(i) - d_i, 0)) = \max_{i=1}^{n}(\max(s(i) + t_i - d_i, 0)).$$

Our goal is to minimize $L(A)$, to minimize the maximum lateness.

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

schedules without idle time

There is no reason to let our resource run idle.

From the code in the Earliest Deadline First algorithm:

$f := 0$
for $i = 1, 2, \ldots, n$ do
    $s(i) := f$
    $f(i) := f + t_i$
    $f := f + t_i$

Observe: request i starts when request $i - 1$ finishes.

Proposition (2)

There is an optimal schedule with no idle time.

In any schedule with "gaps" between requests,
schedule the requests that start after a gap earlier.

an exchange argument

To prove that the output schedule A is optimal,
we start with an optimal schedule $\mathcal{O}$ and transform $\mathcal{O}$ gradually into A.

This type of analysis is an *exchange argument*.

Definition (3)

A schedule has *an inversion* if a job i with deadline d_i is scheduled before another job j with an earlier deadline $d_j < d_i$.

Proposition (4)

All schedules with no inversions and no idle time have the same maximum lateness.

Schedules with no inversions and no idle time can only differ in the order of requests with identical deadlines. Among all requests with identical deadlines, it is the latest that has the greatest lateness.

schedules with no inversions and no idle time

Proposition (4)

All schedules with no inversions and no idle time have the same maximum lateness.

Proof. Consider two different schedules: no inversions, no idle time.

Their difference can only be in the order in which jobs with identical deadlines are scheduled. Let d be the deadline shared by several jobs.

In both schedules, jobs with deadline d are scheduled

- after all jobs with earlier deadlines, and
- before all jobs with later deadlines,

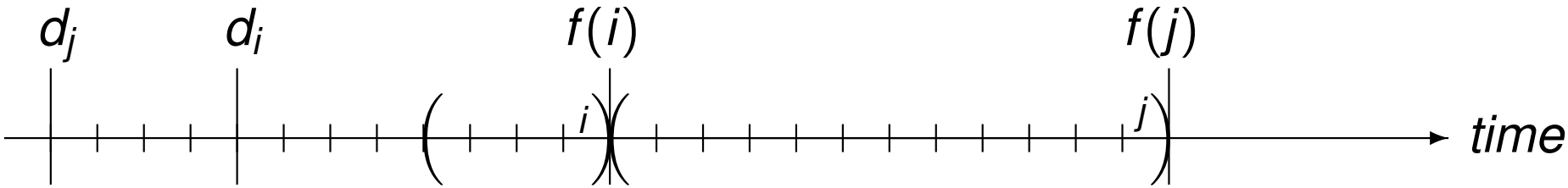
consecutively, in sequence, one after the other.

The last job in this sequence has the maximal lateness.
This lateness does not depend on the order of the jobs.
Therefore, the maximum lateness is the same for both schedules.

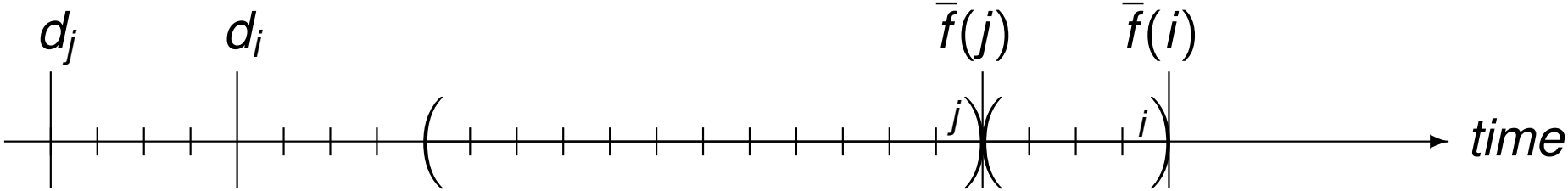
Q.E.D.

swapping an inversion

An inversion:

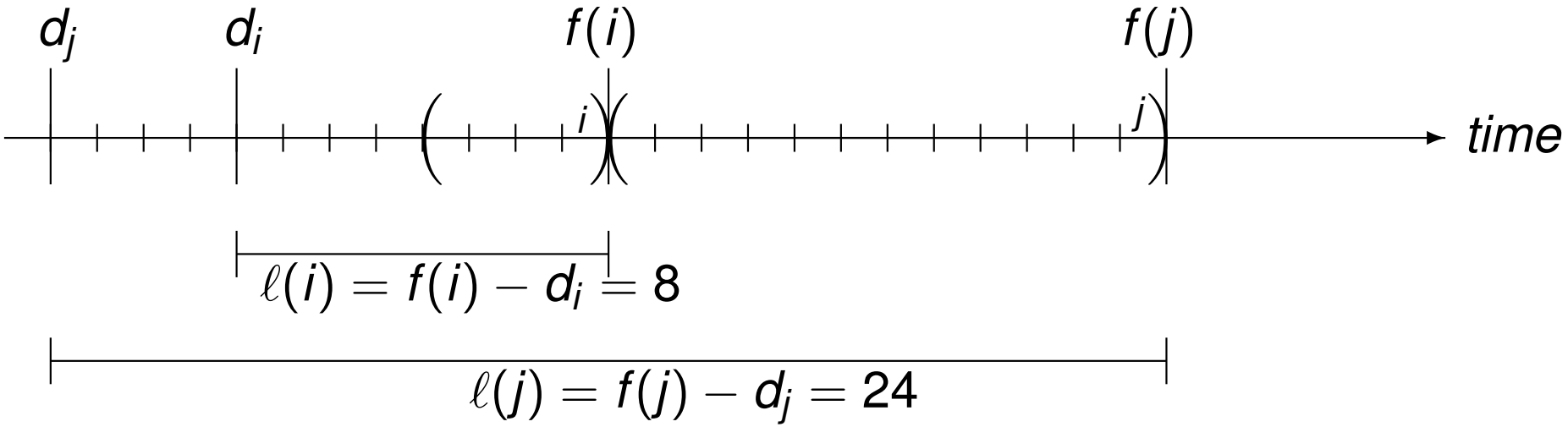


After the swap:

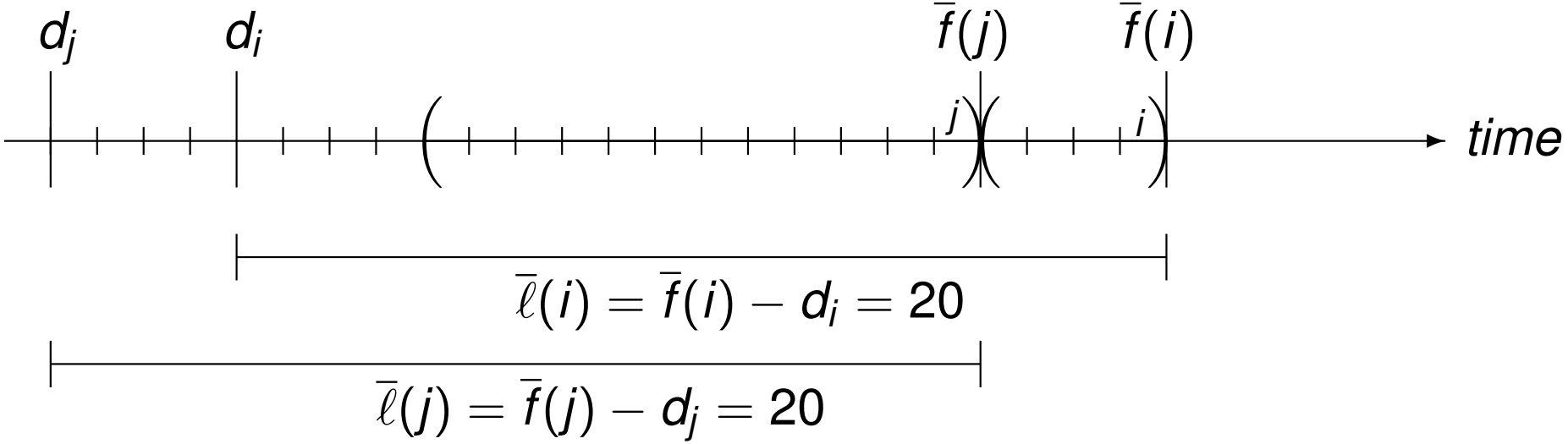


swapping an inversion with lateness computation

An inversion:

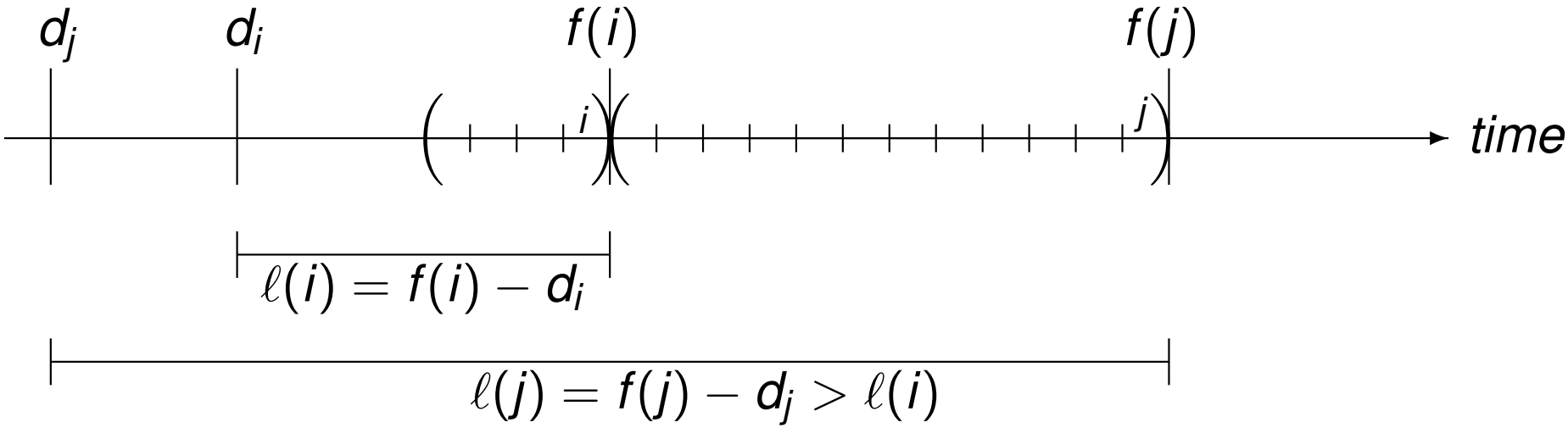


After the swap:

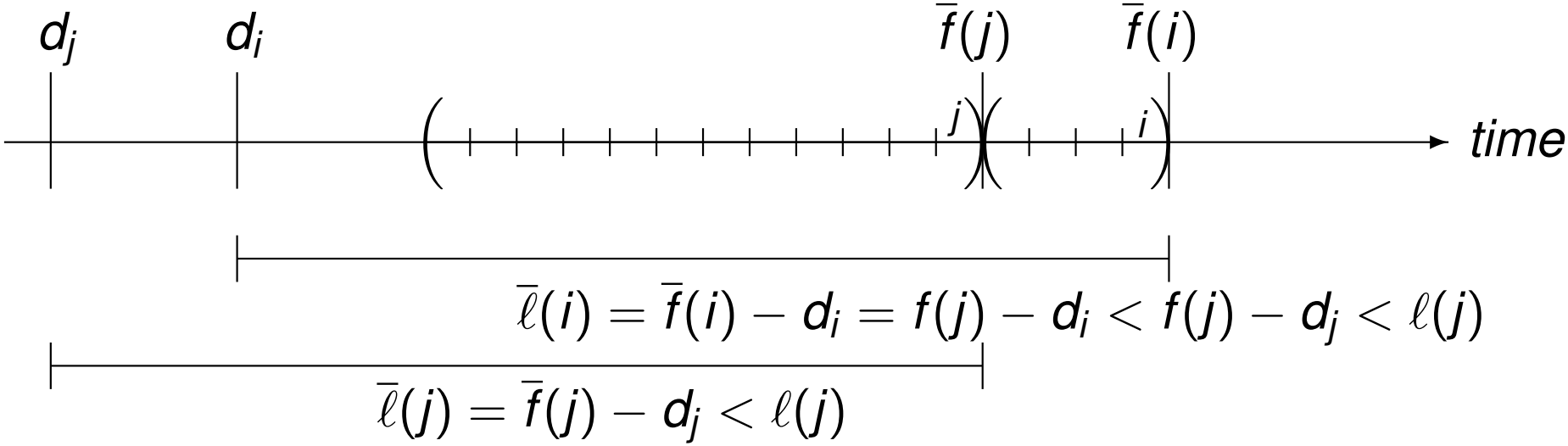


swapping does not increase maximal lateness

An inversion:



After the swap:



swapping does not increase maximal lateness

Lemma (5)

Swapping an inversion for requests scheduled immediately after each other does not increase maximum lateness.

Proof. In an inversion of two requests i and j, we have

- $d_i > d_j$ and $s(i) < s(j)$,
  i has a later deadline and is scheduled sooner,
  j has an earlier deadline and is scheduled later.
- For $\ell(i) = f(i) - d_i$ and $\ell(j) = f(j) - d_j$
  $\ell(j) > \ell(i)$ because $f(j) > f(i)$ and $d_i > d_j$.

After the swap, the new finish time $\bar{f}(i)$ for i equals $f(j)$ because the request are scheduled immediately after each other.
The new lateness for i is $\bar{\ell}(i) = \bar{f}(i) - d_i = f(j) - d_i < f(j) - d_j = \ell(j)$.

Thus, $\overline{\ell(i)} < \ell(j)$ and $\overline{\ell(j)} < \ell(j)$ imply $\max(\overline{\ell(i)}, \overline{\ell(j)}) < \max(\ell(i), \ell(j))$.

The maximum lateness has not increased by the swap. Q.E.D.

the Earliest Deadline First is optimal

Theorem (6)

There is an optimal schedule that has no inversions and no idle time.

Proof. There is an optimal schedule $\mathcal{O}$ with no idle time.

If $\mathcal{O}$ has an inversion, then there is a pair of request i and j such that j is scheduled immediately after i and $d_j < d_i$.

Swapping i and j gives a schedule with one less inversion.

By Lemma (5),

swapping inversions does not increase maximum lateness.

Thus we can transform $\mathcal{O}$ into a schedule
with no inversions and no idle time. Q.E.D.

an alternative metric for lateness

Exercise 2:

In the optimization problem, we defined the lateness of a schedule as the maximum of lateness over all requests.

Consider an alternative metric to measure lateness:

$$L(A) = \sum_{i=1}^{n} \ell(i).$$

Does the Earliest Deadline First algorithm minimize this $L(A)$?

In your answer, discuss the swapping of the inversions with this $L(A)$.

Either prove that the Earliest Deadline First algorithm minimizes also this $L(A)$ or give a numerical counter example.

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

CS 401/MCS 401 Lecture 6
Computer Algorithms I
Jan Verschelde, 29 June 2026

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

the problem of cache maintenance

Memory hierarchies:

1. a small amount of data can be accessed very quickly,
2. a large amount of data needs more time to access.

By *caching* we store a small amount of data in fast memory
to reduce the amount of time spent interacting with slow memory.

A *cache maintenance* algorithm determines

- what to keep in cache, and
- what to evict from the cache when new data is brought in.

A *cache miss* happens when

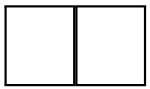
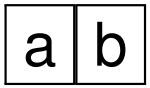
1. the cache is full, and
2. a new piece of data is processed.

a first example

The problem setup is

- $\{ a, b, c \}$, the number of different items is 3
-  , the cache size is 2
- a, b, c, b, c, a, b, the size of the sequence is 7

We run through the items in the sequence:

0.  , a, b, c, b, c, a, b
1.  , b, c, b, c, a, b
2.  , c, b, c, a, b, cache miss

The cache is full with a and b when c is the next item.

1. c will enter the cache, and
2. one element from the cache must be evicted.

the first example, continued to the end

- $\{ a, b, c \}$, the number of different items is 3
- $\square\square$, the cache size is 2
- a, b, c, b, c, a, b, the size of the sequence is 7

We run through the items in the sequence:

0. $\square\square$, a, b, c, b, c, a, b
1. a $\square$, b, c, b, c, a, b
2. a b, c, b, c, a, b, cache miss
3. c b, b, c, a, b, evicted a
4. c b, c, a, b
5. c b, a, b, cache miss
6. a b, b, evicted c
7. a b

Could we have done better?

farthest-in-future algorithm

Given a sequence $D = d_1, d_2, \ldots, d_m$ of data items.

for $i = 1, 2, \ldots, n$ do
    if d_i is not in the cache then
        evict the item from the cache
        that is needed farthest in the future

Definition (7)

An *eviction schedule* determines which items should be evicted from the cache at each point in the sequence of data items.

reduced eviction schedules

An eviction schedule determines which items to evict at a cache miss.

Definition (8)

An eviction schedule is *reduced* if it brings item d in the cache at step i only when there is a request at step i for item d.

Proposition (9)

If S is a nonreduced schedule, then there is a reduced schedule $\widehat{S}$ that brings in at most as many items as schedule S.

The farthest-in-future algorithm evicts an item from the cache that is needed farthest in the future.

The farthest-in-future algorithm is a reduced eviction schedule.

Is the farthest-in-future algorithm optimal?

a second example

Consider the sequence $a, b, c, d, a, d, e, a, d, b, c$ with cache size 3.

0. $[\ ][\ ][\ ]$, $a, b, c, d, a, d, e, a, d, b, c$
1. $[a][\ ][\ ]$, $b, c, d, a, d, e, a, d, b, c$
2. $[a][b][\ ]$, $c, d, a, d, e, a, d, b, c$
3. $[a][b][c]$, d, a, d, e, a, d, b, c, cache miss
4. $[a][b][d]$, a, d, e, a, d, b, c, evicted c
5. $[a][b][d]$, d, e, a, d, b, c
6. $[a][b][d]$, e, a, d, b, c, cache miss
7. $[a][e][d]$, a, d, b, c, evicted b
8. $[a][e][d]$, d, b, c

Observe, instead of c we could have evicted b first,
and then evicted c at the second cache miss, which is also optimal.

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

proving optimality by an exchange argument

We will prove the optimality of the farthest-in-future algorithm by an exchange argument.

Let S^* be an optimal eviction schedule with the minimum number of cache misses.

Let S_{FF} be the schedule of the farthest-in-future algorithm.

If we can transform S^* into S_{FF}, then the farthest-in-future algorithm is optimal.

Theorem (10)

S_{FF} incurs no more misses than any other schedule S^* and hence is optimal.

the base cases for the induction

Theorem (10)

S_{FF} incurs no more misses than any other schedule S^* and hence is optimal.

The proof goes by induction, on the number of steps in the sequence.

Let k be the size of the cache. We start with an empty cache.

In the first k steps, there is a cache miss in each step and we copy the next data element to the cache. There is nothing to evict.

All schedules are the same for the first k steps.

This concludes the discussion of the base cases.

Note that $k \geq 1$.
If k would be larger than the number of data elements in the sequence, there would be nothing left to prove, so sequences are longer than k.

induction step proven by a lemma

Lemma (11)

If there is an optimal reduced schedule S with the same eviction schedule as S_{FF} through the first j requests, then there is optimal reduced schedule S' with the same eviction schedule as S_{FF} through the first $j+1$ requests.

Proof. Consider the $(j+1)$-th request $d = d_{j+1}$.
By the induction hypothesis, S and S_{FF} have the same cache.
Consider the following cases:

1. If d is already in the cache, then $S' = S = S_{FF}$.
2. If d is not in the cache and S and S_{FF} evict the same element, then $S' = S = S_{FF}$.
3. If d is not in the cache, S_{FF} evicts e and S evicts $f \neq e$.

proof continued, third case

3. If d is not in the cache, S_{FF} evicts e and S evicts $f \neq e$.

   Cache for S and S_{FF} at step j: | same | e | f |.

   At step $j+1$, cache for S : | same | e | d |;
   At step $j+1$, cache for S_{FF}: | same | d | f |.

   We will prove that having f in cache does not lead to more cache misses than having e in cache.

   Let step $j' > j+1$ be the first step that S and S' take a different action when g is requested.

   There are three cases to consider:

   1. $g \neq e, g \neq f$
   2. $g = f$
   3. $g = e$

proof continued, g requested at step $j' > j + 1$

3. Cache for S: | same | e | d |; cache for S_{FF}: | same | d | f |.
   1. $g \neq e$, $g \neq f$: because the caches of S and S' are the same, except for e and f, and they take different action, S evicts e and S' evicts f. Now S and S' have the same cache.
   2. $g = f$: if S evicts e, then both caches for S and S' are the same.

      If S evicts $e' \neq e$, then S' evicts e' and brings e in cache.
      Apply Proposition (9), as S' is no longer reduced.

      The caches for S and S' are then the same.
   3. $g = e$ cannot happen with farthest-in-future, because S_{FF} evicted e at step $j + 1$, implying e was needed later than f.

Either we end up with the same cache or otherwise in a situation that does not lead to a different number of cache misses.
So S' can be transformed into S_{FF}. Q.E.D.

the second example revisited

Exercise 3:

Consider the sequence $a, b, c, d, a, d, e, a, d, b, c$ with cache size 3.

In running the farthest-in-future algorithm, we observed there was another eviction schedule S' with the same number of cache misses.

Apply the exchange argument to the example to demonstrate that S' can be transformed into S_{FF}.

cache hierarchies

Exercise 4:

Modern computers have multiple caches between registers and main memory.

Consider two caches where the second cache is at least twice the size of the first cache.

Adapt the farthest-in-future algorithm to work with two caches. Describe the algorithm and illustrate with a numerical example.

Is the adaptation of farthest-in-future to two caches optimal? Either extend the exchange arguments to the double caching, or show why optimality does not hold.

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

CS 401/MCS 401 Lecture 6
Computer Algorithms I
Jan Verschelde, 29 June 2026

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

the shortest path in a weighted directed graph

Let $G = (V, E)$ be a weighted directed graph.
Every edge $(u, v) \in E$ has a length $\ell_{(u,v)}$.

Given a start vertex s,
what is the distance of the shortest path from s to every other vertex?

Designing a greedy algorithm:

1. Assume we have explored a subset $S \subset V$:
   for all $u \in S$: $d(u)$ is the shortest distance from s to u.

2. For $v \in V \setminus S$, if there is some $u \in S$ for which $(u, v) \in E$,
   then set $d(v) := \min_{\substack{u \in S \\ (u,v) \in E}} \left(d(u) + \ell_{(u,v)} \right)$.

The next v to be added to S is the vertex connected to $u \in S$
for which the length of the path is shortest over all possible $u \in S$.

designing a greedy algorithm

Let $G = (V, E)$ be a weighted directed graph.
Every edge $(u, v) \in E$ has a length $\ell_{(u,v)}$.

Given a start vertex s,
what is the distance of the shortest path from s to every other vertex?

$S \subset V$ is the set of explored vertices.

1. For all $u \in S$: $d(u)$ is the shortest distance from s to u.
2. For $v \in V \setminus S$, if there is some $u \in S$ for which $(u, v) \in E$,
   then set $d(v) := \min_{\substack{u \in S \\ (u,v) \in E}} \left(d(u) + \ell_{(u,v)} \right)$.

The length of the shortest path from s to v is then

1. the length $\ell_{(u,v)}$ of the edge (u, v) and
2. $d(u)$ where u is the neighbor to v that is closest to s.

Dijkstra's algorithm

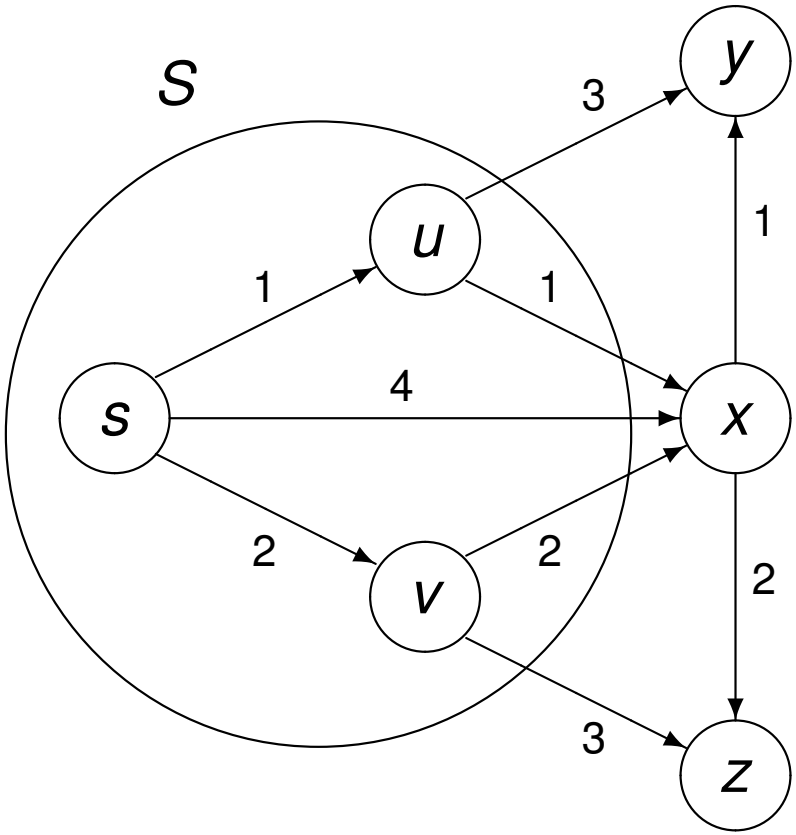
Input: $G = (V, E)$ is a directed graph
with weights $\ell_{(u,v)}$ for all $(u, v) \in E$, and
$s \in V$ is a start vertex.
Output: for all $v \in V$: $d(v)$ is the shortest distance from s to v.

$S := \{ s \}$
$d(s) := 0$
while $S \neq V$ do
for all $v \in V \setminus S$, with $(u, v) \in E$ from some $u \in S$
compute $d'(v) := \min_{\substack{u \in S \\ (u,v) \in E}} \left(d(u) + \ell_{(u,v)} \right)$
select that first v for which $d'(v)$ is minimal
$d(v) := d'(v)$
$S := S \cup \{ v \}$

a snapshot of the execution of Dijkstra's algorithm

$$d'(v) := \min_{\substack{u \in S \\ (u,v) \in E}} \left(d(u) + \ell_{(u,v)} \right)$$

select that first v for which $d'(v)$ is minimal

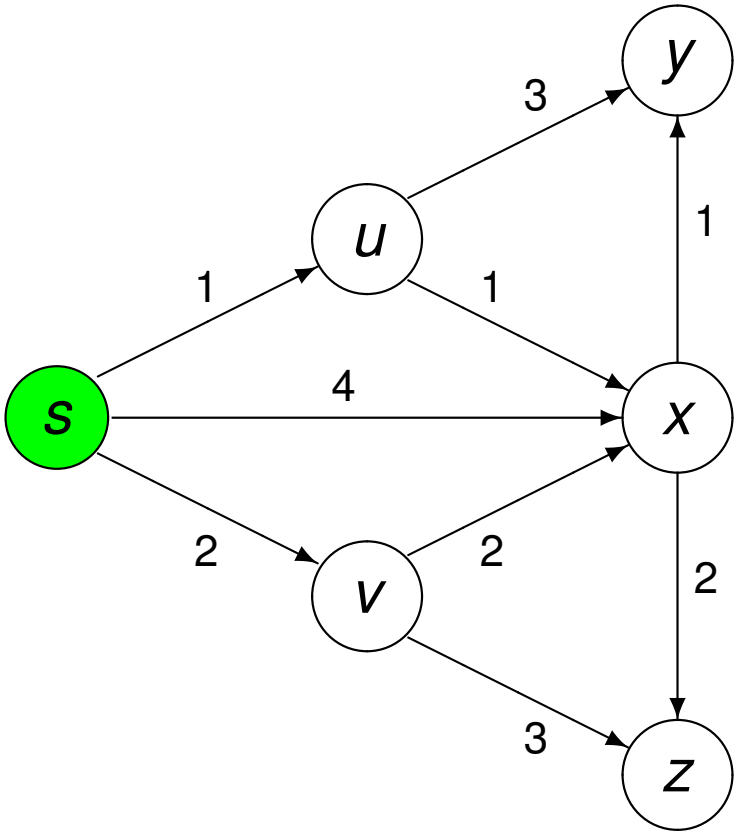


$$d(s) = 0$$
$$d(u) = 1$$
$$d(v) = 2$$

Which node with be added next to S?

step 0 of Dijkstra's algorithm on an example

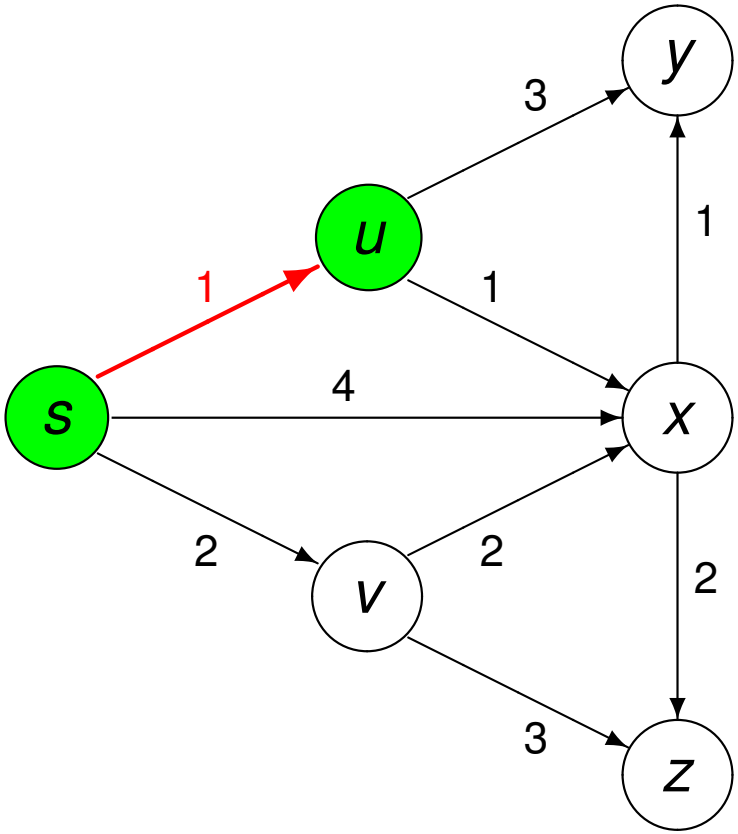
$V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$



$d(s) = 0$

step 1 of Dijkstra's algorithm on an example

$V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$

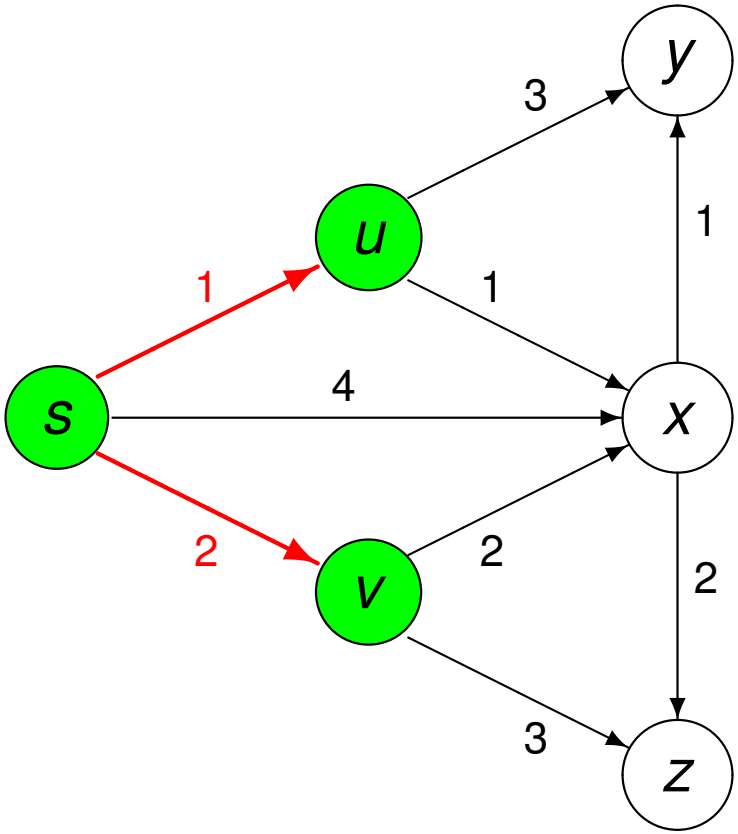


$d(s) = 0$

$d(u) = 1$

step 2 of Dijkstra's algorithm on an example

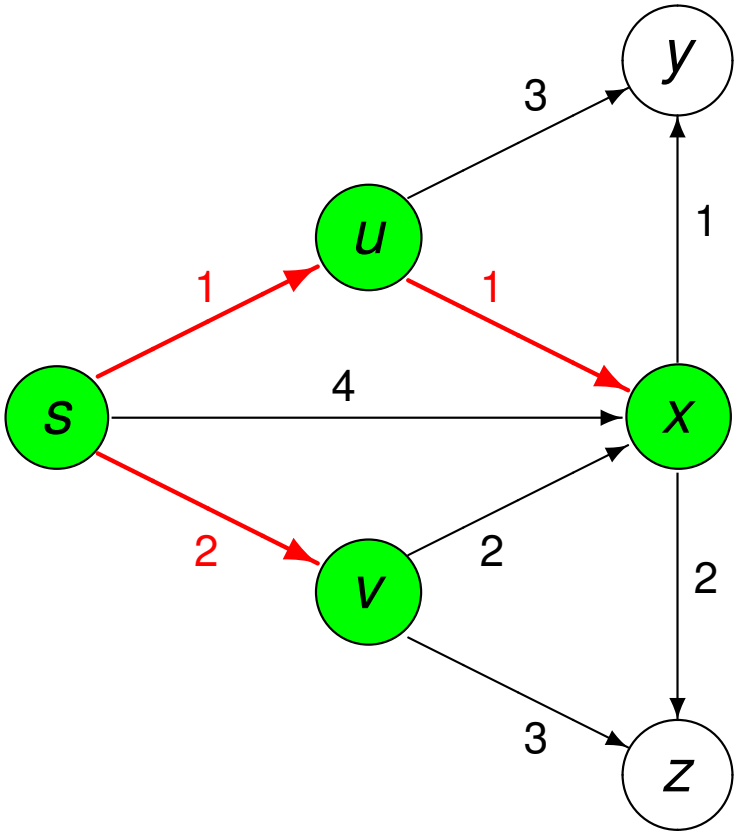
$V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$



$d(s) = 0$
$d(u) = 1$
$d(v) = 2$

step 3 of Dijkstra's algorithm on an example

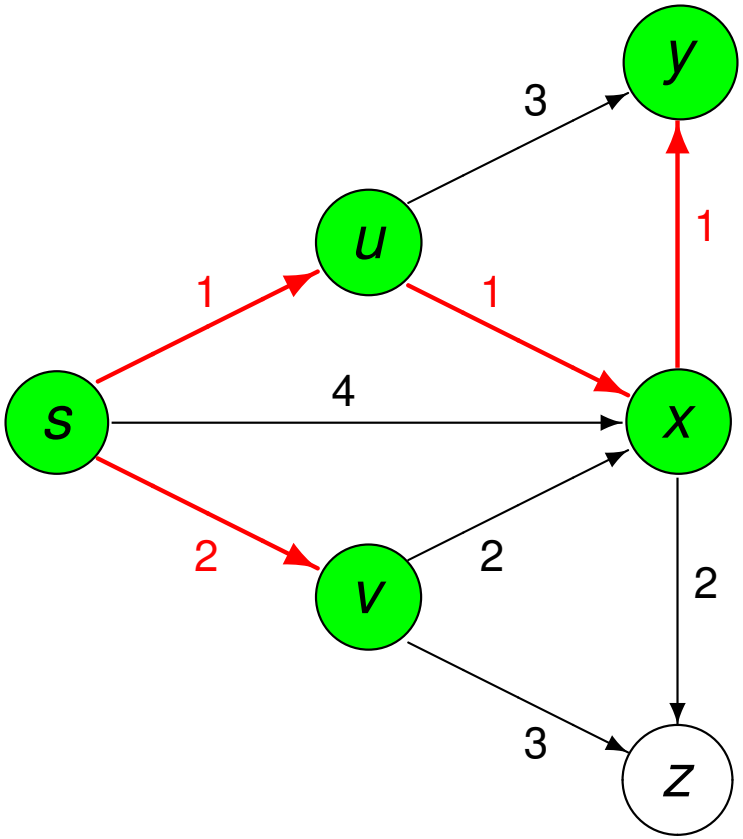
$V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$



$d(s) = 0$
$d(u) = 1$
$d(v) = 2$
$d(x) = 2$

step 4 of Dijkstra's algorithm on an example

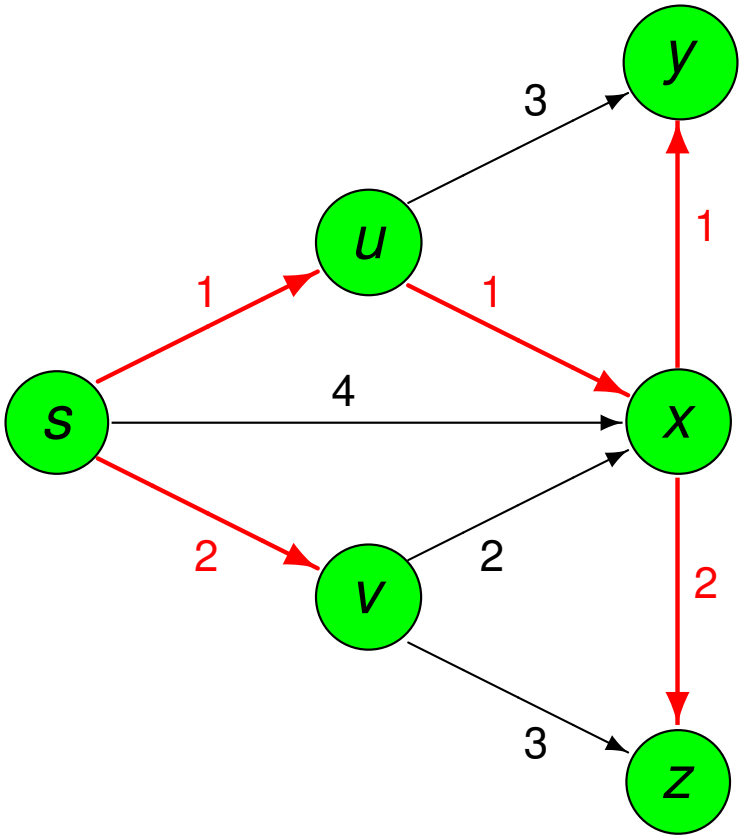
$V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$



$d(s) = 0$
$d(u) = 1$
$d(v) = 2$
$d(x) = 2$
$d(y) = 3$

step 5 of Dijkstra's algorithm on an example

$V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$



$d(s) = 0$
$d(u) = 1$
$d(v) = 2$
$d(x) = 2$
$d(y) = 3$
$d(z) = 4$

termination of Dijkstra's algorithm

Exercise 5:
Prove that Dijkstra's algorithm terminates.

What is the assumption on G that should be made explicit for Dijkstra's algorithm to terminate?

If the assumption on G does not hold,
modify the description of the algorithm so it terminates.

Scheduling, Caching, Shortest Paths

1. Scheduling to Minimize Maximum Lateness
   - problem statement and algorithm
   - swapping inversions
2. Optimal Caching
   - minimizing cache misses
   - proving optimality by an exchange argument
3. Shortest Paths in a Weighted Directed Graph
   - the problem and algorithm
   - Dijkstra's algorithm computes the shortest paths

computing the paths to the start vertex

A modification of Dijkstra's algorithm returns for each vertex v
$P(v)$: the path from s to v.

The paths $P(v)$ can be represented by an array $\mathtt{P}$ of size n:
$\mathtt{P}[v]$ stores the vertex $u \in S$ of the edge (u, v)

for which $d'(v)$ was minimal in $d'(v) := \min_{\substack{u \in S \\ (u,v) \in E}} \left(d(u) + \ell_{(u,v)} \right)$.

Since $u \in S$, $\mathtt{P}[u]$ is already defined.

Via $\mathtt{P}[v]$ we can walk back from v to the start vertex s.

computing the shortest paths

Exercise 6:

Consider the running of Dijkstra's algorithm on the example: $V = \{s, u, v, x, y, z\}$, $\ell_{(s,u)} = 1$, $\ell_{(s,v)} = 2$, $\ell_{(s,x)} = 4$, $\ell_{(u,x)} = 1$, $\ell_{(u,y)} = 3$, $\ell_{(v,x)} = 2$, $\ell_{(v,z)} = 3$, $\ell_{(x,y)} = 1$, $\ell_{(x,z)} = 2$.

1. Illustrate the progress of the vector `P` as the algorithm runs.
2. Show how `P` defines all shortest paths from s to every other vertex in the graph by explicitly listing all paths.

the computed paths are the shortest paths

Theorem (11)

The paths $P(u)$ for all $u \in S$ computed by Dijkstra's algorithm are the shortest paths from the start vertex s to u.

Proof. We prove this by induction on $\#S$.

Base case: $\#S = 1$. In the first step of the algorithm we select the closest neighbor v to s, so $\ell_{(s,v)}$ is minimal.

Induction hypothesis: the theorem is true for all S, $\#S \leq n$, that is:

$$\text{for all } u \in S,\ P(u) \text{ is the shortest path from } s \text{ to } u.$$

Consider the addition of $v \in V \setminus S$ and path $P(v)$.

proving for $\#S = n + 1$ by contradiction

Consider the addition of $v \in V \setminus S$ and path $P(v)$.

Assume $d(v)$ is not the minimal distance from s to v,
then there is another path P for s to v, shorter than $P(v)$,
which must contain a vertex not in S.

Let $x \in S$ be the last vertex in S on the path P
and $y \in V \setminus S$, the first vertex on the path P that is not in S.
By the induction hypothesis, $P(x)$ is the shortest path from s to x.

Because $y \notin S$: $d'(y) = d(x) + \ell_{(x,y)} \geq d(u) + \ell_{(u,v)} = d(v)$.
But as y is on the path P, its length is at least $d'(y) \geq d(v)$,
which contradicts the assumption that there is a shorter path to v.

As $P(v)$ is the shortest path from s to v, and $\#(S \cup \{ v \}) = n + 1$,
the induction hypothesis holds for $n + 1$.

Q.E.D.

running time and implementation

Consider the while loop of Dijkstra's algorithm:

- the loop runs as long as $S \neq V$,
- S is initialized with s,
- in each step $S := S \cup \{ v \}$.

In a graph with n vertices, the loop takes $n - 1$ steps.

In the innermost loop, we consider all edges $(u, v) \in E$.
In a graph with m edges, this loop runs in m steps.

On a graph with n vertices and m edges,
the running time of Dijkstra's algorithm is $O(nm)$.

Using a priority queue to store the vertices,
using the distance $d(v)$ as the key to store the vertex v,
this running time can be improved to $O(m \log(n))$,
where $\log(n)$ is the cost to update the priority queue.